

Sistemas de Recomendación 2026-1

Laboratorio 4

Grupo 1

Participantes:

Lina Muñoz 202310172

Sergio Pérez 202314506

Juan Sebastian Ardila 202110171

Juan Jose Ardila 201511440

Este documento es una extensión del notebook, en el que se documentan las actividades realizadas y se vuelven a responder las preguntas propuestas con mayor detalle.

Preguntas de Interpretación 0

1. ¿Cuál de estas columnas se convierte naturalmente en nodos de un grafo?

userId y movieId. Cada usuario único y cada película única se convierten en nodos de un grafo bipartito: un conjunto de nodos representa usuarios y el otro representa películas. Una arista conecta un usuario con una película cuando existe una interacción (rating) entre ellos

2. ¿Cuál se convierte en peso de una arista?

rating. El valor numérico del rating (de 0.5 a 5.0) representa la intensidad o confianza de la interacción entre un usuario y una película, por lo que se usa como peso de la arista que los conecta. Un rating más alto indica una preferencia más fuerte.

3. ¿Por qué el timestamp será más importante en la parte de transformers que en la parte de PageRank?

PageRank modela la estructura estática del grafo: lo que importa es quién está conectado con quién y con qué peso, sin importar cuándo ocurrió la interacción. El orden temporal no afecta el cálculo de puntajes.

En cambio, el transformer secuencial (SASRec) necesita el timestamp para ordenar las interacciones de cada usuario cronológicamente y construir una secuencia $[i_1, i_2, i_3, \dots]$. El modelo aprende patrones de orden: qué películas suele ver un usuario después de otras. Sin el orden temporal correcto, la secuencia sería arbitraria y el modelo no podría aprender transiciones significativas entre ítems.

Parte 1A

A.1 Tarea 1

El ejercicio parte de un grafo dirigido de cinco nodos (A, B, C, D, E) con las siguientes aristas: $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow D$, $C \rightarrow B$, $C \rightarrow D$, $D \rightarrow E$ y $E \rightarrow C$. En la Figura 1 se visualiza este grafo, donde el nodo A (azul) representa el usuario semilla, los nodos B y C (verde) son nodos ya conocidos, y los nodos D y E (naranja) son los candidatos a recomendar.

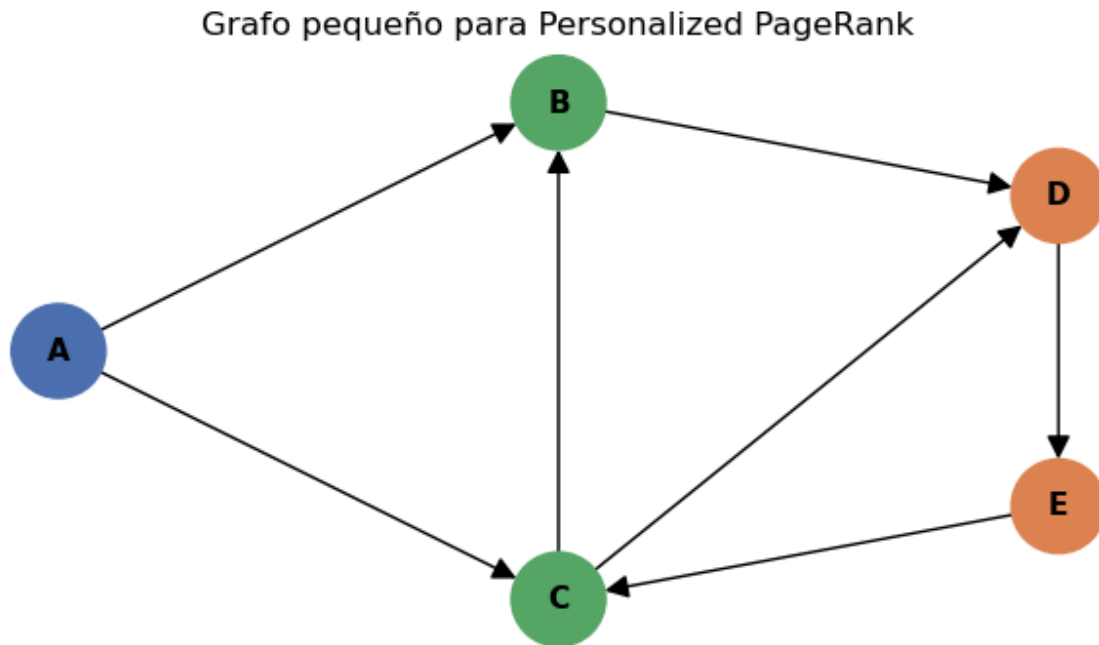


Figura 1. Grafo dirigido de ejemplo para Personalized PageRank

A partir de la estructura del grafo, se construye la matriz de adyacencia siguiendo la convención $A[\text{destino}, \text{origen}] = 1$, donde las columnas representan el nodo de origen y las filas el nodo de destino. Bajo esta convención, la arista $X \rightarrow Y$ se registra en la fila Y, columna X. La Figura 2 muestra la matriz resultante junto con el out-degree de cada nodo (suma por columna), que indica cuántas aristas salen de cada nodo: A tiene 2 salidas, C tiene 2 salidas, y B, D y E tienen 1 salida cada uno

Matriz de adyacencia A: filas=destino, columnas=origen

	A	B	C	D	E
A	0.0	0.0	0.0	0.0	0.0
B	1.0	0.0	1.0	0.0	0.0
C	1.0	0.0	0.0	0.0	1.0
D	0.0	1.0	1.0	0.0	0.0
E	0.0	0.0	0.0	1.0	0.0

Suma por columna = número de salidas de cada nodo

	out_degree
A	2.0
B	1.0
C	2.0
D	1.0
E	1.0

Figura 2. Matriz de adyacencia y out-degree por nodo

Tarea 2

Para obtener la matriz de transición M, cada columna de la matriz de adyacencia se divide entre el out-degree del nodo origen correspondiente, de modo que cada columna sume 1. Esto modela una caminata aleatoria: desde un nodo con k salidas, la probabilidad de transitar hacia cualquiera de sus vecinos es $1/k$. La Figura 3 muestra la matriz de transición resultante y su validación: todos los nodos con aristas salientes presentan una suma de columna igual a 1.0, confirmando que la normalización es correcta. Nótese que el nodo A tiene probabilidad 0.5 hacia B y 0.5 hacia C; el nodo C distribuye 0.5 hacia B y 0.5 hacia D; y el nodo E transfiere toda su masa hacia C con probabilidad 1.0

Matriz de transición M:

	A	B	C	D	E
A	0.0	0.0	0.0	0.0	0.0
B	0.5	0.0	0.5	0.0	0.0
C	0.5	0.0	0.0	0.0	1.0
D	0.0	1.0	0.5	0.0	0.0
E	0.0	0.0	0.0	1.0	0.0

Validación: cada columna con salidas debe sumar 1

column_sum	
A	1.0
B	1.0
C	1.0
D	1.0
E	1.0

Figura 3. Matriz de transición normalizada por columnas y validación de sumas

Tarea 3

La Figura 4 muestra la configuración del vector de personalización p y el vector inicial r_0 . Dado que el nodo semilla es A, toda la masa de probabilidad se concentra en dicho nodo ($p[A] = 1.0$), mientras que los demás nodos parten con puntaje cero. Este vector cumple dos roles: define el punto de partida de la iteración ($r_0 = p$) y actúa como ancla de teleportación en cada paso, controlada por el factor $(1 - \alpha) = 0.15$. Se utiliza $\alpha = 0.85$, valor estándar que balancea la exploración del grafo con el retorno al nodo semilla

```

alpha = 0.85
p = pd.Series(0.0, index=example_nodes)
p.loc["A"] = 1.0

r = p.copy()
print("Vector inicial r_0 = p")
display(r.to_frame("r_0"))

```

Vector inicial r_0 = p

	r_0
A	1.0
B	0.0
C	0.0
D	0.0
E	0.0

Figura 4. Vector de personalización y estado inicial r_0

Tarea 4

La Figura 5 presenta la evolución del vector de puntajes r a lo largo de 3 iteraciones, aplicando la fórmula $r_{t+1} = 0.85 \cdot M \cdot r_t + 0.15 \cdot p$. En r_1 , la probabilidad fluye directamente desde A hacia sus vecinos inmediatos B y C, cada uno con puntaje 0.425, mientras que D y E permanecen en cero al no ser alcanzables en un solo salto. En r_2 , la masa continúa propagándose: D recibe flujo desde B y C y alcanza el puntaje más alto de la iteración (0.5419), mientras que B y C disminuyen. En r_3 , E emerge como el nodo con mayor puntaje (0.4606) al recibir toda la masa de D a través de la arista $D \rightarrow E$, superando a D (0.2348). El nodo A se estabiliza en 0.15 en todas las iteraciones, valor igual a $(1 - \alpha)$, dado que ningún otro nodo apunta hacia él.

Iteraciones con $\alpha=0.85$				
	r_0	r_1	r_2	r_3
A	1.0	0.150	0.1500	0.1500
B	0.0	0.425	0.2444	0.0908
C	0.0	0.425	0.0638	0.0638
D	0.0	0.000	0.5419	0.2348
E	0.0	0.000	0.0000	0.4606

Figura 5. Tabla de puntajes por nodo en cada iteración de Personalized PageRank

Tarea 5

Para generar la recomendación, se excluyen del ranking el nodo semilla A y los nodos ya conocidos B y C, dejando únicamente los candidatos D y E. La Figura 6 muestra los puntajes finales tras 3 iteraciones y el nodo recomendado. El nodo E obtiene un puntaje de 0.4606, casi el doble que D (0.2348), por lo que se recomienda E. Esto se explica por la estructura del grafo: D es el único nodo que apunta hacia E con arista $D \rightarrow E$, y a su vez D recibe flujo acumulado desde B y C. Al cabo de 3 iteraciones, esta cadena de transiciones concentra suficiente masa de probabilidad en E para posicionarlo como el candidato más relevante desde la perspectiva del nodo semilla A.

Candidatos ordenados por puntaje final:	
	score_after_3_iterations
E	0.460594
D	0.234812
Nodo recomendado: E	

Figura 6. Candidatos ordenados por puntaje final tras 3 iteraciones y nodo recomendado

A1.2

1. ¿Qué ocurre si α baja de 0.85 a 0.50?

Al reducir alpha a 0.50, la mitad de la probabilidad en cada iteración se redistribuye de vuelta al nodo semilla A en lugar de fluir por las aristas del grafo. El resultado es que todos los nodos distintos de A convergen a puntajes más bajos y más similares entre sí, porque el grafo tiene menos influencia. Las recomendaciones se vuelven menos sensibles a la estructura de conexiones y más dominadas por la personalización inicial. Con alpha muy bajo (cerca de 0), el ranking casi no distingue entre candidatos

2. ¿Por qué la recomendación puede cambiar si aumentamos el número de iteraciones?

Con pocas iteraciones, la probabilidad solo ha viajado un número limitado de saltos desde el nodo semilla. Nodos lejanos (como E, que requiere pasar por $A \rightarrow B/C \rightarrow D \rightarrow E$) reciben poca o ninguna masa en las primeras iteraciones. Al aumentar las iteraciones, la probabilidad sigue propagándose y redistribuyendo por ciclos (ej. $E \rightarrow C \rightarrow D \rightarrow E$), lo que puede hacer que un nodo alejado acumule más puntaje que uno más cercano. La recomendación es estable solo cuando el vector r converge

3. ¿Qué error cometemos si normalizamos filas pero usamos la fórmula diseñada para columnas?

Si se normalizan filas (cada fila suma 1), la matriz M ya no representa probabilidades de transición desde cada origen hacia sus destinos: ahora cada fila dice "con qué probabilidad llego a este nodo destino desde cada posible origen", que no tiene interpretación coherente. Al multiplicar $M * r$, se estarían sumando contribuciones de todos los orígenes ponderadas por el vector r actual, pero en la dirección incorrecta. El flujo de probabilidad viaja en sentido opuesto al grafo real: un nodo con muchas entradas parecería "distribuir" probabilidad en lugar de recibirla. El resultado numérico puede converger, pero recomendaría nodos equivocados.

Parte 1B

Actividad 1.1

	TOP_K_USERS	TOP_M_MOVIES	Usuarios	Peliculas	Aristas
0	10	50	10	50	457
1	20	100	20	100	1638
2	50	200	50	200	6486
3	100	500	100	500	20767
4	200	1000	200	1000	44209

Figura 7: Cambios en el grafo bipartito por cambio en parámetros.

Para analizar el comportamiento del grafo bipartito entre usuarios y películas, se probaron diferentes configuraciones de TOP_K_USERS y TOP_M_MOVIES. A medida que se aumentaron estos valores, el número de aristas creció rápidamente, debido a que cada usuario puede estar conectado con múltiples películas mediante ratings. Los resultados obtenidos mostraron que con 10 usuarios y 50 películas se generaron 457 aristas, mientras que con 200 usuarios y 1000 películas se alcanzaron más de 44 mil conexiones.

Este crecimiento no fue lineal. Al incrementar el número de usuarios y películas, las conexiones aumentaron de forma acelerada porque el número de relaciones posibles dentro del grafo también crece significativamente. Como consecuencia, el grafo empezó a volverse visualmente más saturado y computacionalmente más costoso de procesar.

Las configuraciones pequeñas permitieron visualizar claramente las relaciones entre usuarios y películas, facilitando la interpretación del grafo. Por ejemplo, configuraciones como TOP_K_USERS = 10 y TOP_M_MOVIES = 50 generaron una representación limpia y fácil de analizar. Sin embargo, estas configuraciones reducen considerablemente la cantidad de información disponible del dataset original.

La configuración intermedia de TOP_K_USERS = 50 y TOP_M_MOVIES = 200 representó un buen equilibrio entre cantidad de información y claridad visual. El grafo aún era manejable y permitía identificar patrones relevantes sin generar una saturación extrema de nodos y aristas.

A partir de configuraciones más grandes, especialmente desde TOP_K_USERS = 100 y TOP_M_MOVIES = 500, el rendimiento empezó a verse afectado. El aumento del número de aristas implicó un mayor consumo de memoria y tiempos de ejecución más altos durante la construcción y visualización del grafo. Con la configuración más grande utilizada, el grafo se volvió mucho más difícil de interpretar visualmente debido a la enorme cantidad de conexiones presentes.

Actividad 1.2

Usando todos los ratings					
	rank	movielfid	title	genres	pagerank_score
0	1	2858	American Beauty (1999)	Drama Romance	0.002754
1	2	1265	Groundhog Day (1993)	Comedy Fantasy Romance	0.002613
2	3	2716	Ghostbusters (a.k.a. Ghost Busters) (1984)	Action Comedy Sci-Fi	0.002442
3	4	1197	Princess Bride, The (1987)	Action Adventure Comedy Fantasy Romance	0.002429
4	5	2174	Beetlejuice (1988)	Comedy Fantasy	0.002123
Usando solo ratings positivos (≥ 4.0)					
	rank	movielfid	title	genres	pagerank_score
0	1	2858	American Beauty (1999)	Drama Romance	0.002764
1	2	1265	Groundhog Day (1993)	Comedy Fantasy Romance	0.002609
2	3	2716	Ghostbusters (a.k.a. Ghost Busters) (1984)	Action Comedy Sci-Fi	0.002437
3	4	1197	Princess Bride, The (1987)	Action Adventure Comedy Fantasy Romance	0.002426
4	5	4878	Donnie Darko (2001)	Drama Mystery Sci-Fi Thriller	0.002111

Figura 8: Recomendaciones usando el filtro vs no usándolo.

Se compararon las recomendaciones obtenidas utilizando Personalized PageRank con y sin el filtro de ratings positivos ($\text{rating} \geq 4.0$). En ambos casos, varias películas aparecieron en posiciones similares dentro del ranking, incluyendo películas como American Beauty, Groundhog Day y Ghostbusters. Esto indica que dichas películas tienen una fuerte relevancia dentro del grafo y se encuentran altamente conectadas con los gustos del usuario.

El Personalized PageRank funciona mediante una caminata aleatoria sobre el grafo. El algoritmo inicia desde las películas asociadas al usuario y recorre conexiones hacia otros usuarios y películas relacionadas. Durante este proceso, las películas que reciben más visitas acumulan un mayor pagerank_score, indicando una mayor relevancia para el usuario.

Cuando se utilizó el filtro de ratings positivos, el algoritmo sólo consideró películas calificadas con valores mayores o iguales a 4.0. Esto hizo que la caminata se concentrará únicamente en películas que realmente le gustaron al usuario, eliminando señales provenientes de películas con bajas calificaciones. Como resultado, las recomendaciones se volvieron más coherentes con las preferencias reales del usuario.

El valor de pagerank_score representa la relevancia relativa de cada película dentro del grafo. Un score más alto indica que la película está más conectada con los gustos del usuario y tiene una mayor probabilidad de ser recomendada. Estos valores son pequeños porque representan probabilidades normalizadas dentro de toda la red, pero lo importante es la comparación relativa entre películas.

Actividad 1.3

Baseline de popularidad para userId=18						
rank	movieid	title		genres	num_ratings	avg_rating
37	1	1265	Groundhog Day (1993)	Comedy Fantasy Romance	42	3.869048
50	2	2628	Star Wars: Episode I - The Phantom Menace (1999)	Action Adventure Sci-Fi	42	3.023809
54	3	2858	American Beauty (1999)	Drama Romance	40	4.262500
51	4	2716	Ghostbusters (a.k.a. Ghost Busters) (1984)	Action Comedy Sci-Fi	40	3.800000
56	5	2987	Who Framed Roger Rabbit? (1988)	Adventure Animation Children Comedy Crime Fantasy	37	3.567568
32	6	1197	Princess Bride, The (1987)	Action Adventure Comedy Fantasy Romance	36	4.208333
45	7	2174	Beetlejuice (1988)	Comedy Fantasy	36	3.680556
30	8	1097	E.T. the Extra-Terrestrial (1982)	Children Drama Sci-Fi	35	3.728571
22	9	592	Batman (1989)	Action Crime Thriller	35	3.500000
46	10	2355	Bug's Life, A (1998)	Adventure Animation Children Comedy	35	3.471429

Figura 9: Recomendación a usuario 18 en base a popularidad

Para este punto se usó un baseline de popularidad para comparar los resultados de Personalized PageRank con un enfoque más simple. Este método recomienda películas basándose únicamente en la cantidad de ratings y el promedio de calificaciones recibidas globalmente, excluyendo películas ya vistas por el usuario.

Al comparar ambos métodos, se encontró que 8 películas aparecieron simultáneamente en ambos top-10. Entre ellas estuvieron Groundhog Day, American Beauty, Ghostbusters y The Princess Bride. Esto demuestra que varias películas populares también poseen una alta relevancia estructural dentro del grafo.

Sin embargo, Personalized PageRank mostró un comportamiento más personalizado, ya que utiliza las conexiones entre usuarios y películas para encontrar recomendaciones relacionadas específicamente con los gustos del usuario. El baseline de popularidad, en cambio, recomienda películas populares globalmente sin considerar preferencias individuales.

Con respecto a interpretabilidad, el baseline de popularidad resulta mucho más sencillo de explicar a un usuario final, ya que simplemente recomienda películas populares y altamente calificadas por la mayoría de personas. Personalized PageRank requiere explicar conceptos más complejos como grafos, conexiones y caminatas aleatorias.

Recomendaciones Personalized PageRank para userId=18 con alpha=0.65					
	rank	movieId	title	genres	pagerank_score
0	1	2858	American Beauty (1999)	Drama Romance	0.001803
1	2	1265	Groundhog Day (1993)	Comedy Fantasy Romance	0.001702
2	3	2716	Ghostbusters (a.k.a. Ghost Busters) (1984)	Action Comedy Sci-Fi	0.001590
3	4	1197	Princess Bride, The (1987)	Action Adventure Comedy Fantasy Romance	0.001579
4	5	4878	Donnie Darko (2001)	Drama Mystery Sci-Fi Thriller	0.001388
5	6	2174	Beetlejuice (1988)	Comedy Fantasy	0.001378
6	7	2987	Who Framed Roger Rabbit? (1988)	Adventure Animation Children Comedy Crime Fant...	0.001376
7	8	1097	E.T. the Extra-Terrestrial (1982)	Children Drama Sci-Fi	0.001362
8	9	1968	Breakfast Club, The (1985)	Comedy Drama	0.001360
9	10	2628	Star Wars: Episode I - The Phantom Menace (1999)	Action Adventure Sci-Fi	0.001333

Figura 10: Recomendación a usuario 18 con Personlized PageRank co alpha 0.65

También se evaluó el impacto del parámetro alpha sobre el ranking de recomendaciones. Cuando se utilizó un valor menor como $\alpha = 0.65$, la caminata aleatoria reinició con mayor frecuencia desde las películas favoritas del usuario. Esto produjo recomendaciones más locales y enfocadas en preferencias específicas.

Recomendaciones Personalized PageRank para userId=18 con alpha=0.91					
	rank	movieId	title	genres	pagerank_score
0	1	2858	American Beauty (1999)	Drama Romance	0.003059
1	2	1265	Groundhog Day (1993)	Comedy Fantasy Romance	0.002907
2	3	2716	Ghostbusters (a.k.a. Ghost Busters) (1984)	Action Comedy Sci-Fi	0.002718
3	4	1197	Princess Bride, The (1987)	Action Adventure Comedy Fantasy Romance	0.002706
4	5	2174	Beetlejuice (1988)	Comedy Fantasy	0.002365
5	6	2987	Who Framed Roger Rabbit? (1988)	Adventure Animation Children Comedy Crime Fant...	0.002357
6	7	4878	Donnie Darko (2001)	Drama Mystery Sci-Fi Thriller	0.002333
7	8	1097	E.T. the Extra-Terrestrial (1982)	Children Drama Sci-Fi	0.002332
8	9	1968	Breakfast Club, The (1985)	Comedy Drama	0.002331
9	10	2628	Star Wars: Episode I - The Phantom Menace (1999)	Action Adventure Sci-Fi	0.002273

Figura 11: Recomendación a usuario 18 con Personlized PageRank co alpha 0.91

Por otro lado, al aumentar el valor a $\alpha = 0.91$, la caminata exploró regiones más lejanas del grafo antes de reiniciar. Como consecuencia, las recomendaciones comenzaron a parecerse más a películas populares o altamente conectadas dentro de la red. Aun así, varias películas permanecieron estables en las primeras posiciones debido a su fuerte relevancia tanto global como personalizada dentro del grafo.

Parte 2

Pregunta 2.1 — Remapeo de movieId a índices consecutivos

¿Por qué remapear movieId a índices consecutivos (item_idx)? Pista: piensa en una capa Embedding. ¿Sería eficiente crear una matriz de embeddings indexada por los IDs originales de MovieLens?

La capa nn.Embedding reserva memoria para todas las posiciones desde 0 hasta el índice máximo recibido, sin importar si estos índices se usan o no. En MovieLens los movieId originales llegan a valores superiores a 193.000 y, además, presentan huecos (no son contiguos), lo que obligaría a instanciar una matriz de embeddings de aproximadamente $193.000 \times d_{\text{model}}$ con la mayoría de filas vacías y sin gradiente útil.

Al remapear a índices compactos en el rango $[1, N]$, la matriz de embeddings queda con $N+1$ filas (reservando el índice 0 para el token de padding), una por cada item presente en el vocabulario. Esto reduce drásticamente el uso de memoria, acelera el lookup, evita parámetros muertos y produce una matriz cuyas filas son todas entrenables con datos reales.

Actividad 2.2 — Relación input/target en predicción causal

Observa $x_{\text{batch}}[0]$ y $y_{\text{batch}}[0]$. ¿Qué relación hay entre ambas secuencias? ¿Qué posición intenta predecir el modelo en cada paso? ¿Por qué ignoramos PAD_IDX en la función de pérdida?

Relación entre input y target

El target y es la secuencia x desplazada un paso hacia la izquierda. Si la secuencia original de un usuario es [A, B, C, D], tras aplicar padding por la izquierda quedan:

x (input) : [PAD, PAD, A, B, C]

y (target) : [PAD, PAD, B, C, D]

En cada posición t, el modelo recibe el ítem actual como entrada y debe predecir el siguiente ítem de la secuencia. Este es el objetivo causal estándar de SASRec: predicción del próximo token a partir del historial.

¿Qué predice el modelo en cada posición?

En la posición t, gracias a la máscara causal, el modelo solo puede atender a las posiciones $0..t$, y su objetivo es producir un puntaje (logit) sobre el vocabulario completo que ranquee correctamente al ítem en la posición $t+1$. La última posición de la secuencia es la más informativa porque concentra todo el historial disponible para predecir el ítem inmediatamente siguiente, y es la que se utiliza en inferencia para generar recomendaciones.

¿Por qué se ignora PAD_IDX en la pérdida?

Las posiciones de padding son relleno artificial introducido para igualar el tamaño de los tensores dentro del batch; no representan interacciones reales del usuario. Si no se ignoraran en

CrossEntropyLoss, el modelo aprendería a predecir el token PAD en esas posiciones, lo que es una tarea trivial y abundante en el dataset. Esto produciría dos efectos indeseados: el loss se verían artificialmente bajos y la señal de aprendizaje real (predecir ítems en posiciones válidas) quedaría diluida. Por eso se utiliza `ignore_index=PAD_IDX`, que elimina la contribución de esas posiciones al gradiente.

Actividad 2.3 — Máscara causal

La máscara causal evita que la posición actual observe ítems futuros. (1) ¿Qué ocurriría si el modelo pudiera ver el futuro durante entrenamiento? (2) ¿Por qué eso produciría una evaluación demasiado optimista? (3) ¿Qué posición adicional se bloquea al cambiar $diagonal=1$ por $diagonal=0$?

1. Consecuencia de ver el futuro en entrenamiento

El modelo haría trampa: en la posición t , en lugar de inferir el ítem $t+1$ a partir del historial, podría simplemente copiar el valor desde su entrada (porque la entrada contiene la secuencia completa). El loss bajaría casi a cero sin que el modelo aprenda ningún patrón secuencial real. Este fenómeno se conoce como label leakage o fuga del objetivo.

2. ¿Por qué la evaluación quedaría sobre-optimista?

Durante inferencia el futuro no existe: solo se dispone del historial hasta el momento actual. Por lo tanto, un modelo entrenado con fuga jamás aprendió a inferir desde el pasado y su rendimiento real colapsa frente al rendimiento aparente. El gap entre el loss de entrenamiento y métricas como `HitRate@K` en datos no vistos sería enorme. La evaluación quedaría sobre optimista porque mide una capacidad que el modelo en realidad no tiene.

3. Efecto de $diagonal=0$

Con $diagonal=1$, `torch.triu` produce una matriz triangular superior estricta donde la diagonal principal queda en cero (no bloqueada), permitiendo que cada posición se atienda a sí misma. Al cambiar a $diagonal=0$, la diagonal principal también se bloquea: la posición t pierde la capacidad de usar el embedding del ítem actual y solo puede atender a los ítems estrictamente anteriores. Esto degrada el aprendizaje porque el ítem presente en la posición t es la señal más relevante para construir la representación que proyectará el siguiente ítem; bloquearlo equivale a obligar al modelo a predecir el ítem $t+1$ sin conocer el ítem t , lo que es estrictamente más difícil que la formulación estándar.

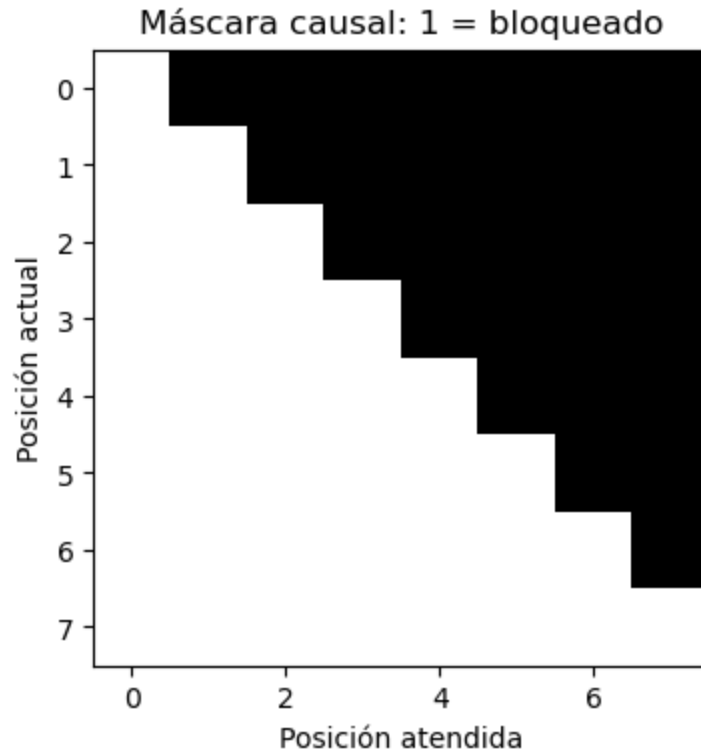


Figura 11: Visualización de la máscara causal ($\text{seq_len}=8$): las celdas negras representan las posiciones bloqueadas que evitan que el modelo atienda al futuro.

Actividad 2.5 — Interpretación de la evaluación con holdout temporal

1. ¿Por qué $\text{HitRate}@20$ suele ser mayor o igual que $\text{HitRate}@10$?
2. ¿Qué limitaciones tiene evaluar solo con el último ítem de cada usuario?
3. ¿Qué cambiarías para una evaluación más robusta?

1. ¿Por qué $\text{HitRate}@20 \geq \text{HitRate}@10$?

$\text{HitRate}@K$ mide si el ítem objetivo aparece dentro de los K mejores candidatos rankeados por el modelo. Al ampliar la lista de $K=10$ a $K=20$, todos los hits que ya estaban en el top-10 siguen estando en el top-20 (el top-10 es subconjunto del top-20), y además pueden sumarse nuevos hits que el modelo puntuó entre las posiciones 11 y 20. Por construcción, $\text{HitRate}@20 \geq \text{HitRate}@10$ siempre.

2. Limitaciones de evaluar solo con el último ítem

- Una sola oportunidad por usuario: si el modelo falla ese único ítem, el usuario cuenta como miss completo aunque otras predicciones intermedias hubieran sido buenas.

- Alta sensibilidad al azar: si el último rating del usuario corresponde a un ítem poco popular o atípico (un outlier de su gusto), la métrica castiga al modelo de forma desproporcionada.
- No mide el historial intermedio: el modelo puede tener un buen desempeño promedio prediciendo posiciones intermedias y aun así obtener un HitRate bajo si el último ítem particular es difícil.

3. Mejoras para una evaluación más robusta

- Leave-last-N-out: reservar los últimos N ítems (no solo el último) para validación, evaluando el modelo en múltiples puntos del historial y promediando.
- Negative sampling controlado: en lugar de rankear contra todo el vocabulario, seleccionar un conjunto fijo de negativos por usuario para hacer la evaluación reproducible y comparable entre experimentos.
- Métricas complementarias: agregar NDCG@K y MRR para capturar no solo si el ítem está en el top-K sino en qué posición aparece.

... Recomendaciones transformer para userId=1

...	rank	movieId	title	genres
295	1	337	What's Eating Gilbert Grape (1993)	Drama
2078	2	2762	Sixth Sense, The (1999)	Drama Horror Mystery
418	3	480	Jurassic Park (1993)	Action Adventure Sci-Fi Thriller
2248	4	2985	RoboCop (1987)	Action Crime Drama Sci-Fi Thriller
2195	5	2918	Ferris Bueller's Day Off (1986)	Comedy
514	6	597	Pretty Woman (1990)	Comedy Romance
20	7	21	Get Shorty (1995)	Comedy Crime Thriller
512	8	595	Beauty and the Beast (1991)	Animation Children Fantasy Musical Romance IMAX
1183	9	1580	Men in Black (a.k.a. MIB) (1997)	Action Comedy Sci-Fi
5344	10	8910	I Heart Huckabees (2004)	Comedy

Figura 12: Tabla Top-10 recomendaciones del transformer TinySASRec para *userId=1* tras excluir los ítems ya vistos por el usuario.

Actividad 2.4 — Experimento con hiper parámetros

Cambia uno de los hiper parámetros (*d_model*, *nhead*, *num_layers* o *MAX_SEQ_LEN*) y vuelve a entrenar. Registra número de parámetros, tiempo aproximado de entrenamiento, forma de la curva de pérdida y cambios cualitativos en las recomendaciones.

Configuración del experimento

Se comparó la configuración base (num_layers=2) contra una variante con el doble de capas transformer (num_layers=4), manteniendo constantes el resto de hiper parámetros: d_model=64, nhead=4, seq_len=40, 5 épocas y 220 batches por época.

Resultados

Métrica	Base (num_layers=2)	num_layers=4
Loss final (época 5)	13.62	7.50
Loss inicial (época 1)	184	126
HitRate@10	0.008	0.025
Tiempo total de entrenamiento	≈ 113 s	≈ 264 s
Parámetros entrenables	Base (~200 K)	+50 % (~100 K adicionales)

Interpretación de los resultados

El modelo con cuatro capas converge a un loss final de 7.50 frente a 13.62 del modelo base, casi la mitad, lo que indica una capacidad de aprendizaje sustancialmente mayor. Esta diferencia es visible desde la primera época (126 vs 184), sugiriendo que la mayor profundidad no solo permite alcanzar un mejor mínimo, sino que también acelera el descenso inicial del loss.

En términos de métrica de recomendación, el HitRate@10 pasa de 0.008 a 0.025, es decir, aproximadamente tres veces mejor al duplicar el número de capas. Sin embargo, el costo computacional escala de forma proporcional: el tiempo de entrenamiento se duplica (113 s → 264 s) y los parámetros entrenables aumentan alrededor de un 50 %.

Conclusión: aumentar la profundidad reduce el loss de forma significativa y mejora las métricas downstream, pero a un costo de tiempo proporcional. En CPU este impacto es notorio incluso con un dataset pequeño; en producción con GPU el incremento sería más asumible y justificaría explorar configuraciones aún más profundas.

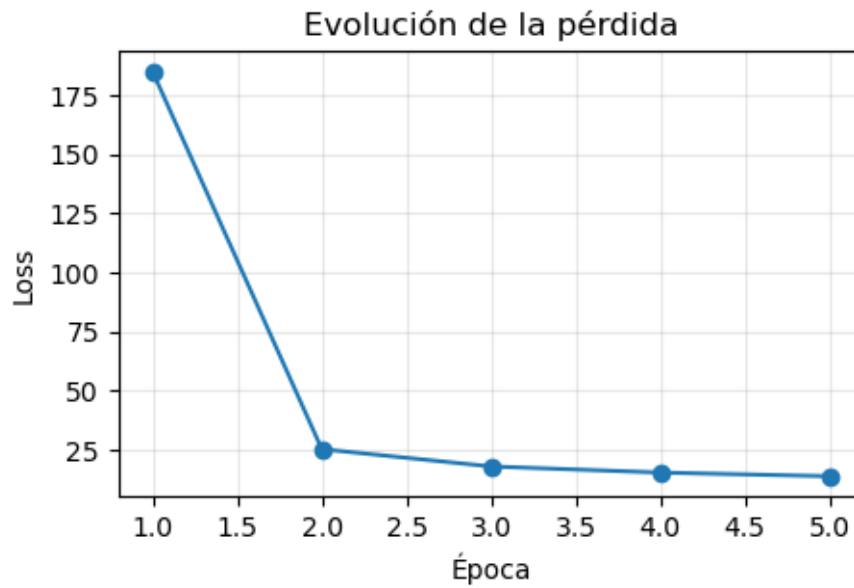


Figura 13: Curva de pérdida del modelo base TinySASRec a lo largo de 5 épocas de entrenamiento.

Preguntas de cierre

Teniendo en cuenta lo aprendido en el laboratorio, e investigación externa se responde:

1. ¿En qué dominio usarías primero PageRank y por qué?

En un dominio donde las relaciones entre entidades importan más que el orden temporal, como recomendación de artículos académicos o noticias. Un paper citado por muchos papers relevantes para ti es probablemente importante, sin importar cuándo fue publicado. PageRank también es ideal cuando el historial por usuario es escaso (cold-start parcial), porque aprovecha la conectividad global del grafo en lugar de requerir una secuencia larga por usuario.

2. ¿En qué dominio esperarías que un transformer secuencial aporte más valor?

En e-commerce o streaming de video/música, donde el contexto reciente domina: lo que alguien busca hoy puede no tener nada que ver con lo que buscó hace seis meses. El transformer captura ese cambio de intención dentro de la sesión. También en dominios con vocabularios de items grandes y dinámicos (nuevos productos constantemente), donde la señal de orden es clave para distinguir intención.

3. ¿Qué método sería más fácil d

PageRank. Una vez calculados los scores offline, las recomendaciones se sirven con una simple consulta a una tabla ordenada. No requiere GPU, no necesita reentrenamiento frecuente, y su latencia en inferencia es casi cero. El transformer requiere mantener un modelo en memoria, potencialmente GPU para latencia aceptable, y reentrenamiento periódico para no quedar desactualizado.

4. ¿Qué información adicional añadirías al modelo: géneros, tags, texto, precios, contexto de sesión?

1. Géneros y tags: permiten generalizar entre ítems con poco historial (cold-start de ítem). Si el modelo nunca vio una película nueva, sus géneros permiten aproximar su embedding.
2. Contexto de sesión: hora del día y dispositivo cambian la intención (películas cortas en móvil vs. series largas en TV por la noche).
3. Texto (sinopsis, reseñas): útil para capturar semántica que el ID no tiene — dos thrillers psicológicos sin usuarios en común podrían tener embeddings similares si usamos texto.

Conclusiones

Este laboratorio permitió comprender cómo dos enfoques distintos de sistemas de recomendación, basados en grafos y en modelos secuenciales, abordan el problema de recomendar ítems desde perspectivas complementarias. Personalized PageRank mostró cómo las relaciones estructurales dentro de un grafo bipartito permiten identificar películas relevantes a partir de conexiones entre usuarios e ítems, incluso sin considerar el orden temporal de las interacciones. A través de los experimentos realizados, se evidenció que parámetros como α y el número de iteraciones afectan directamente la propagación de probabilidad y el comportamiento de las recomendaciones, permitiendo controlar qué tan local o global es la exploración dentro del grafo.

Por otro lado, la implementación del modelo secuencial tipo SASRec permitió analizar el impacto de los transformers en sistemas de recomendación modernos. Se observó que el orden temporal de las interacciones es fundamental para capturar patrones de comportamiento y predecir el siguiente ítem de un usuario. Conceptos como embeddings, máscaras causales y evaluación temporal mostraron cómo estos modelos dependen fuertemente de una correcta representación secuencial de los datos. Además, el experimento variando el número de capas transformer evidenció el trade-off existente entre desempeño y costo computacional: modelos más profundos logran mejores métricas y menor loss, pero requieren significativamente más tiempo y recursos para entrenar.

Finalmente, el laboratorio permitió reflexionar sobre las ventajas y limitaciones de cada enfoque dependiendo del dominio de aplicación. Mientras PageRank resulta más interpretable, eficiente y sencillo de desplegar en producción, los transformers secuenciales ofrecen una mayor capacidad para modelar cambios de intención y comportamientos dinámicos en contextos como e-commerce o plataformas de streaming. En conjunto, ambas aproximaciones muestran cómo los sistemas de recomendación modernos combinan técnicas de teoría de grafos, aprendizaje profundo y modelado secuencial para generar recomendaciones cada vez más precisas y personalizadas.